

ARCHITECTURE DESIGN

GENOMIC SEQUENCE CLASSIFICATION

Architecture design for pangenome-to-DNABERT prediction and the gene classification web application.

M.TECH PROJECT | MENTOR EVALUATION
IMPLEMENTATION-BASED DESIGN | VERSION 1.0
16 OCTOBER 2026 | RESEARCH PROTOTYPE

CONTENTS

1. EXECUTIVE SUMMARY AND SCOPE 3
Problem statement and objectives 3
2. OVERALL SYSTEM ARCHITECTURE 4
Architecture style and boundaries 5
3. PANGENOME PROCESSING ARCHITECTURE 5
4. DNABERT MODEL LIFECYCLE 7
Input representation and classification 7
Training and saved-model inference 8
5. CLASSICAL ML APPLICATION ARCHITECTURE 8
Feature and model pipeline 9
6. INTERFACES, DATA AND OWNERSHIP 10
Input contracts 10
State and persistence boundaries 11
7. EXPERIMENTAL EVALUATION DESIGN 11
Integrity controls and remaining gaps 12
8. DEPLOYMENT AND QUALITY ATTRIBUTES 13
Current deployment topology 13
Operational prerequisites 14
9. RATIONALE AND EVOLUTION ROADMAP 15
Prioritized future architecture 15
Architectural contribution 16
10. IMPLEMENTATION MAP AND EVALUATION CHECKLIST 16
Mentor demonstration checklist 17
Verification boundary 17

OVERVIEW

1. EXECUTIVE SUMMARY AND SCOPE

The project implements two complementary but independent genomic classification workflows. The first converts a pangenome graph into linear DNA sequences and uses a DNABERT sequence classifier to predict benign or pathogenic labels. The second provides a browser interface for training and evaluating classical machine-learning classifiers on APP, PSEN1 and PSEN2 sequences, followed by uploaded FASTA prediction. This document gives both workflows equal architectural coverage and describes the implementation inspected on 16 October 2026.

The architecture is a **local, modular research prototype**, not an integrated clinical platform. The two workflows share the domain of DNA sequence classification, but use different inputs, feature representations, label semantics and execution entry points. No current request path connects the Flask application to DNABERT, and no graph-to-FASTA dataset bridge is implemented.

PROBLEM STATEMENT AND OBJECTIVES

The engineering objective is to transform genomic inputs into reproducible classification experiments while separating preprocessing, model fitting, inference and evaluation. The graph workflow explores how graph-derived sequence contexts can be represented for a pretrained DNA language model. The web workflow makes sequence-feature experiments and model comparisons accessible through a visual interface. Scientific validity depends on independently verified annotations and biologically meaningful train/test separation.

Dimension	Pangenome / DNABERT	Web / classical ML
Entry point	main.py command-line pipeline	app.py Flask UI; gene_classifier.py CLI
Input	GFA/rGFA subset; optional CSV/TSV/VCF labels	Gene-organized FASTA files; prediction upload
Representation	Ordered overlapping k-mer token IDs	K-mer frequencies and engineered features

Dimension	Pangenome / DNABERT	Web / classical ML
Output classes	0 = BENIGN; 1 = PATHOGENIC	0 = Healthy; 1 = Unhealthy
Persistence	Saved checkpoint, tokenizer and experiment.json	Active web pipeline in process memory

Table 1. Scope and boundaries of the implemented workflows.

SOURCE: MAIN.PY; APP.PY; GENE_CLASSIFIER.PY; VALIDATION_PIPELINE.PY.

> Research use only. A sequence-level model output is not a diagnosis or a verified variant-level clinical interpretation. Healthy/Unhealthy folder labels must not be treated as automatically equivalent to curated benign/pathogenic annotations. No predictive performance or clinical validity is established by this architecture review.

SYSTEM VIEW

2. OVERALL SYSTEM ARCHITECTURE

Each workflow has its own controller and model lifecycle. Local files provide the primary data boundary, and trained artifacts or JSON responses provide the output boundary. The DNABERT workflow can load a local checkpoint or retrieve public pretrained artifacts when network access is available. The browser interface uses Bootstrap, Font Awesome and Chart.js assets loaded from external CDNs.

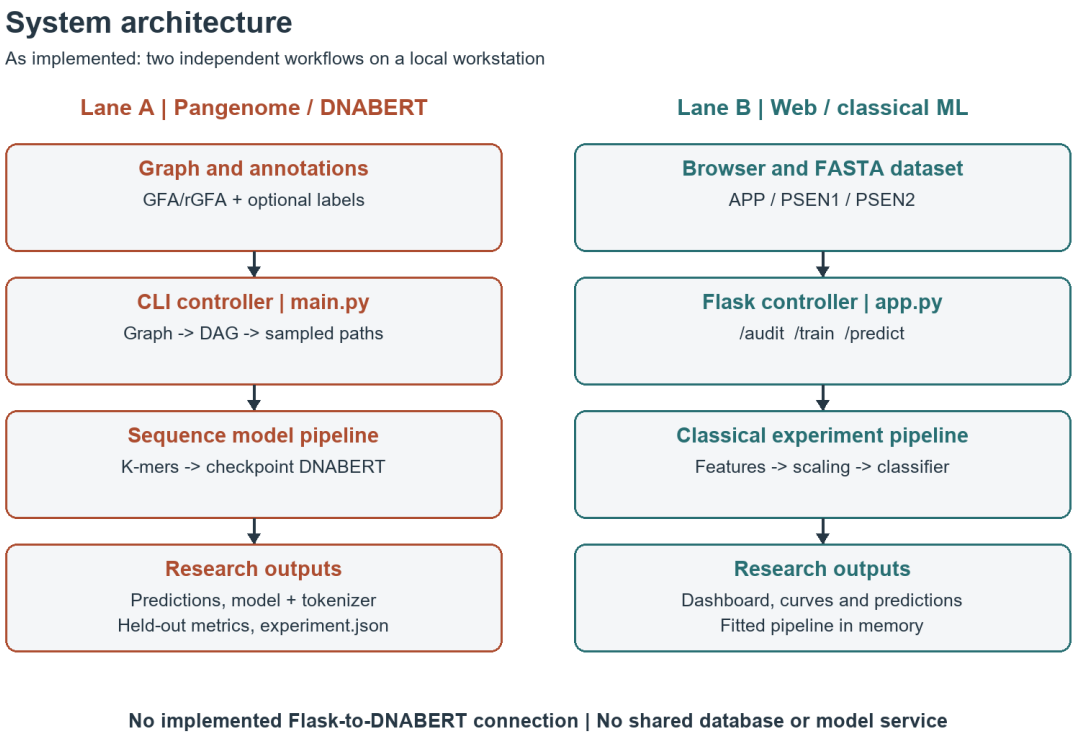


Figure 1. As-built system context with two independent execution lanes. Arrows denote implemented data flow, not a shared prediction service.

MAIN.PY IMPORTS AND ORCHESTRATION; APP.PY ROUTES; TEMPLATES/INDEX.HTML BROWSER REQUESTS.

ARCHITECTURE STYLE AND BOUNDARIES

The command-line workflow is a staged batch pipeline with explicit Python module boundaries. The web workflow is a single-process client/server application with synchronous training requests and an in-memory trained pipeline. There is no implemented database, message queue, authentication service, model registry or background training worker. These components are not required for a local demonstration, but would be necessary design considerations for shared or production deployment.

GRAPH WORKFLOW

3. PANGENOME PROCESSING ARCHITECTURE

The graph workflow is coordinated by main.py and configured through PipelineConfig. Graph parsing, path extraction, sequence construction and model preparation are separate stages. Its final classifier consumes

sequence tokens, not graph adjacency, so it is a graph-derived sequence classification pipeline rather than a graph neural network.

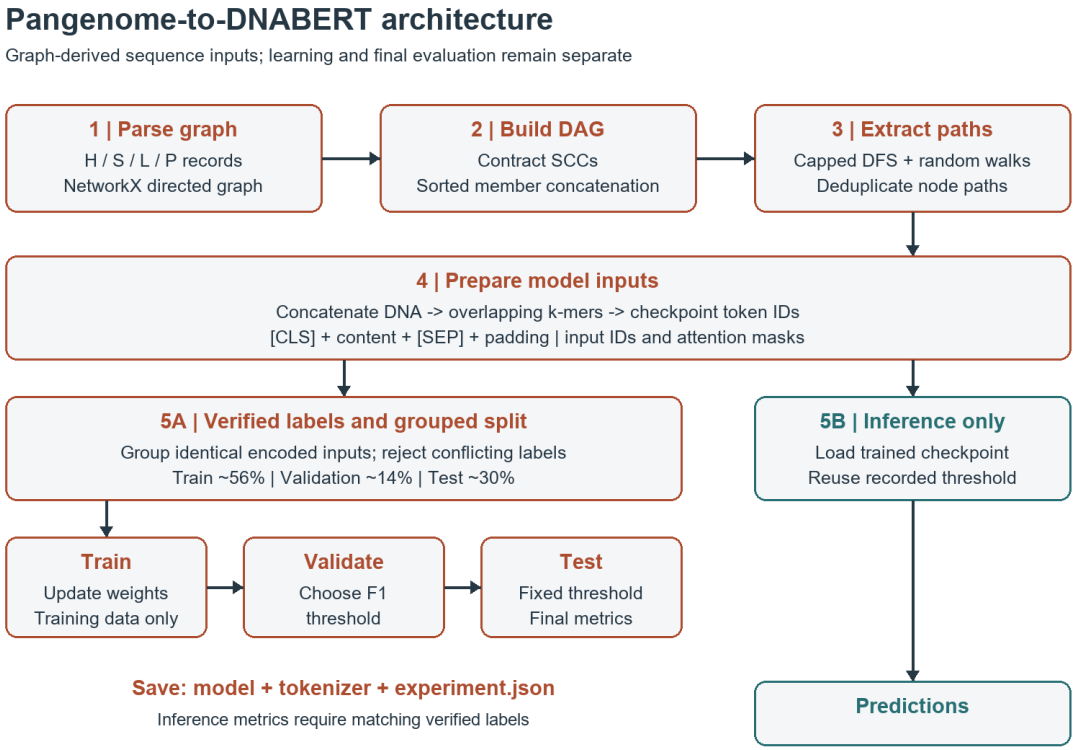


Figure 2. Graph-to-DNABERT processing and the separated training, validation and test branches.

GRAPH_PROCESSING.PY; PATH_EXTRACTION.PY; SEQUENCE_PROCESSING.PY; TOKENIZER_MODULE.PY; MAIN.PY; DNABERT_MODEL.PY.

Graph processing reads H, S, L and P records from a pragmatic GFA/rGFA subset. Segments become NetworkX nodes and links become directed edges. Strongly connected components are contracted into super-nodes to obtain a DAG; member sequences are concatenated in sorted identifier order. Path extraction then combines capped source-to-sink depth-first enumeration with seeded random walks and removes duplicate node paths.

> Biological fidelity is limited: sorted SCC concatenation need not be a valid biological traversal; sequence linearization does not apply reverse complements or remove overlap bases. Named P paths are parsed but are not used by the extraction controller. These are implementation limitations, not properties of a full GFA reconstruction algorithm.

TRANSFORMER

4. DNABERT MODEL LIFECYCLE

INPUT REPRESENTATION AND CLASSIFICATION

For sequence length n and stride-one k-mer size k, the token count is max(0, n - k + 1). Token IDs are taken from the selected checkpoint tokenizer, not from an independently assigned vocabulary ordering. Encoding produces [CLS], at most L - 2 k-mers, [SEP] and padding to length L. The model receives integer input IDs and attention masks shaped [batch, L]; stored position IDs are not passed explicitly, so BERT uses its internal position handling.

The implementation wraps Hugging Face BertForSequenceClassification. The BERT encoder and classification head produce two logits, which are converted to softmax probabilities. A fixed decision threshold on the pathogenic probability determines the reported class. The wrapper selects CUDA when available, otherwise CPU; it rejects incompatible vocabulary sizes, invalid position lengths and inference with a newly initialized classification head.

Setting	Implemented default
Checkpoint and k	zhihan1996/DNA_bert_3; k = 3
Encoded length	128 tokens, including special tokens
Path extraction budget	10 enumerated paths + 3 sampled walks before deduplication
Training	10 epochs; batch size 8; AdamW learning rate 0.00002
Data separation	30% test groups; 20% of remaining groups for validation

Setting	Implemented default
Threshold selection	Validation F1 over 0.05 to 0.95 in 0.05 steps; ties prefer 0.5

Table 2. DNABERT defaults and selection policy.

SOURCE: MAIN.PY PARSE_ARGS, SPLIT_LABELLED_INPUTS AND SELECT_VALIDATION_THRESHOLD; DNABERT_MODEL.PY TRAIN_MODEL.

TRAINING AND SAVED-MODEL INFERENCE

Labels are mapped to extracted sequence indices. Identical encoded token-ID vectors are grouped before stratified splitting; conflicting labels for the same encoded input stop the run. The default group fractions are approximately 56% train, 14% validation and 30% test, although sample fractions vary with group sizes. Training uses class-weighted cross-entropy and only the training partition updates model weights.

Validation selects the threshold after fixed-epoch training, and the held-out test partition supplies final metrics. Training saves the checkpoint, tokenizer and experiment.json with configuration, package versions, input hashes, split indices, class counts, history, threshold and metrics. Inference reuses the saved threshold when present and checks k and encoded length against that record. Without labels, no accuracy or F1 is calculated.

> At the default L = 128 and k = 3, only the first 128 bases are represented by 126 overlapping k-mers. Longer sequences generate a truncation warning. Variant-centred windows, long-sequence aggregation and automatic coordinate-based annotation matching are not implemented.

WEB WORKFLOW

5. CLASSICAL ML APPLICATION ARCHITECTURE

The browser submits settings to Flask and displays dataset summaries, cross-validation comparisons, held-out metrics, ROC/precision-recall curves and feature importance when available. Flask delegates experiment construction to validation_pipeline.py and stores the resulting fitted scikit-learn Pipeline in _state. Uploaded predictions reuse that fitted pipeline, including the same reference sequences and preprocessing.

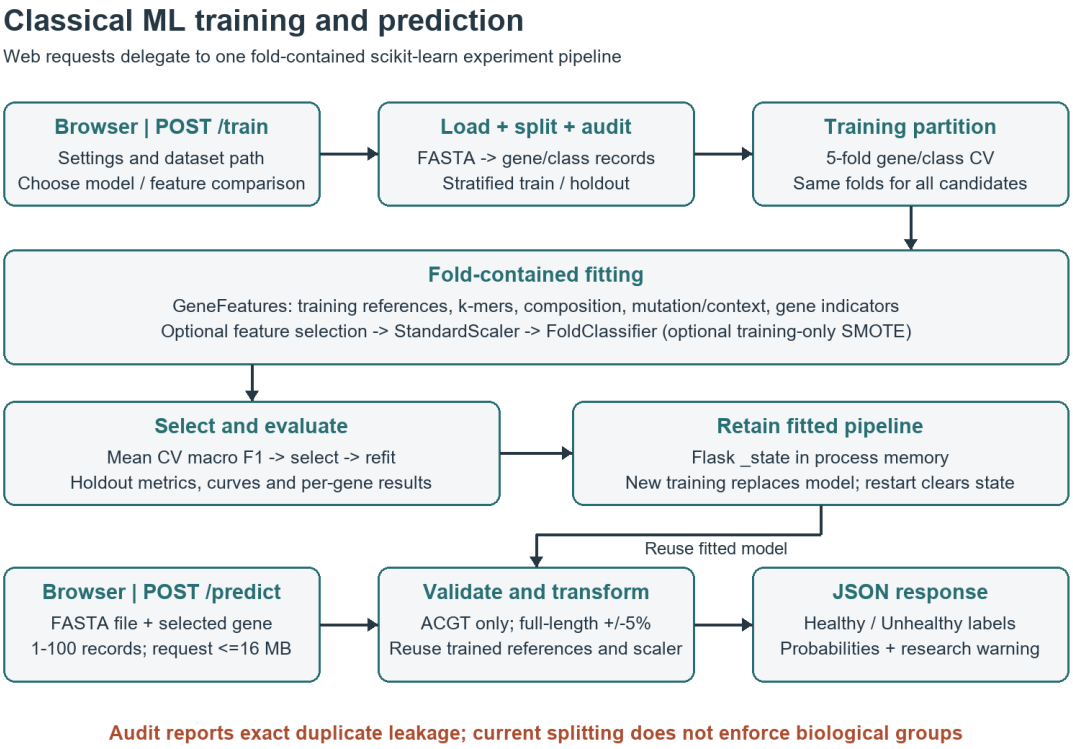


Figure 3. Web training and prediction flow. Selection uses training-fold results; the holdout is evaluated only after the winner is chosen.

APP.PY; VALIDATION_PIPELINE.PY RUN_EXPERIMENT, GENEFEATURES AND FOLDCLASSIFIER; GENE_CLASSIFIER.PY PREPROCESSING.

FEATURE AND MODEL PIPELINE

Feature families include normalized global k-mer frequencies, six nucleotide-composition statistics, eleven mutation/position descriptors, local-context k-mer frequencies and three gene indicators. With k = 4, the all-feature representation contains 256 + 6 + 11 + 256 + 3 = 532 features. Healthy reference sequences are learned inside each training fold, rather than from held-out inputs.

The fitted pipeline is GeneFeatures, optional SelectKBest, StandardScaler and FoldClassifier. Optional SMOTE runs inside FoldClassifier.fit, so validation/test observations are not resampled. The web model choices are Random Forest, SVM, Logistic Regression, Gradient Boosting or comparison mode. Comparison mode also includes XGBoost/LightGBM if installed; tuning, feature selection and soft voting are broader engine/CLI options, not separate web controls.

CONTRACTS

6. INTERFACES, DATA AND OWNERSHIP

Interface	Input and responsibility	Output
GET /	Browser page request	Rendered index.html
POST /audit	JSON dataset path; exact-sequence duplicate/conflict audit	Audit JSON or 400 error
POST /train	JSON dataset, kmer, model, test_size, smote, compare_features	Metrics, comparisons, plots and split metadata
POST /predict	Multipart FASTA file and trained gene identifier	Per-record class, optional probability and research warning

Table 3. Implemented Flask interfaces.

SOURCE: APP.PY ROUTE HANDLERS AND RESPONSE DICTIONARIES.

INPUT CONTRACTS

The classical dataset loader expects APP, PSEN1 and PSEN2 folders, each containing output_safe_healthy and output_safe_unhealthy FASTA subfolders. Class labels are derived from these folder names. The training loader retains A/C/G/T characters and concatenates sequence lines, so source cleaning and file-level record semantics must be verified before scientific evaluation.

Prediction uploads are stricter: a FASTA header is required; 1 to 100 records are accepted; every base must be A, C, G or T; the selected gene must have a trained reference; and each sequence must be within +/-5% of that reference length. The application limits the complete request to 16 MB. These checks constrain input shape but do not verify gene identity or coordinate alignment.

The DNABERT annotation loader accepts integer sequence/path IDs in CSV/TSV and label aliases mapped to 0 or 1. VCF labels are indexed by variant record order. Neither file format establishes that those indices

identify the extracted graph paths. The project researcher must verify this correspondence and preserve it when extraction settings change.

STATE AND PERSISTENCE BOUNDARIES

The graph, DAG, paths, sequences and encoded inputs are in-memory intermediates. The DNABERT experiment directory is the durable model/evaluation artifact boundary. In the web application, the trained pipeline lives in one process-wide dictionary: the next training request replaces it and a restart clears it. Existing model files elsewhere in the project are not automatically loaded by this web route. A relational schema is therefore not part of the current design.

VALIDATION

7. EXPERIMENTAL EVALUATION DESIGN

The two workflows require separate performance statements until they are evaluated on a common, verified biological cohort with equivalent endpoints. The classical workflow makes a gene-plus-class stratified holdout split, then uses five-fold stratification within the training partition. Candidate model/feature combinations use the same folds; the winner is selected by mean cross-validation macro F1, refitted on the full training partition and evaluated on the holdout.

Measure	Purpose and interpretation
Accuracy	Overall correctness; insufficient alone under class imbalance.
Precision / recall	False-positive trade-off and detection of the positive class; report each class in the web workflow.
F1	DNABERT reports binary F1; classical evaluation reports macro and weighted F1. These are not interchangeable.
ROC-AUC / AP	Probability ranking metrics. Average precision is not trapezoidal PR-AUC.

Measure	Purpose and interpretation
Confusion matrix	Counts in fixed [0, 1] label order, with the workflow-specific class definitions.
Per-gene / CV spread	Web workflow reports gene-specific holdout metrics and training-fold means/standard deviations.

Table 4. Evaluation outputs and reporting guidance.

SOURCE: MAIN.PY STAGE_EVALUATION; DNABERT_MODEL.PY EVALUATE_PREDICTIONS; VALIDATION_PIPELINE.PY SCORE_PREDICTIONS AND RUN_EXPERIMENT.

INTEGRITY CONTROLS AND REMAINING GAPS

DNABERT keeps identical encoded inputs within one partition, but this does not group related patients, biological families or mutation-generation sources. The classical audit reports exact duplicate sequences, conflicting labels and overlap across the holdout; it does not group them during splitting or automatically block contaminated experiments. Five-fold fitting contains learned references, scaling, feature selection and optional resampling, but a contaminated dataset can still yield misleading scores.

For mentor evaluation, report verified dataset provenance, source/family identifiers, class counts, split policy, seed, software versions and confidence intervals. Compare sequence-only baselines and graph-derived contexts only after controlling the cohort, labels, splits and sequence windows. Use an independent external cohort before making generalization claims. Model probabilities in either workflow are not calibrated disease-risk estimates.

> No experiment was rerun and no measured accuracy is asserted in this document. Existing checkpoints, plots, logs and labels were not treated as evidence of biological validity. The evaluation protocol above separates existing controls from additional research requirements.

RUNTIME

8. DEPLOYMENT AND QUALITY ATTRIBUTES

CURRENT DEPLOYMENT TOPOLOGY

The intended demonstration topology is one workstation containing the browser, Flask process, dataset folders and Python environment, plus a separately launched DNABERT command-line process. The Flask development entry point listens on port 5000 with debug enabled. Training is synchronous and can keep a request occupied for a long time. DNABERT uses local CPU or available CUDA hardware; no distributed training or hosted model endpoint is implemented.

Attribute	Current design and implication	Next control
Reproducibility	Seeded splits; DNABERT input hashes and package versions.	Persist classical experiment metadata and dataset provenance.
Performance	Capped graph paths; bounded token length; local model training.	Measure latency, peak memory and truncation coverage on real inputs.
Reliability	Explicit failures for invalid splits, checkpoint mismatch and untrained head.	Add run isolation, durable results and structured training job status.
Scalability	In-memory NetworkX graph and process-local web model.	Profile first; add isolated workers and artifact storage if required.

Attribute	Current design and implication	Next control
Privacy / access	Local research files; no implemented user identity boundary.	Keep local; add access controls and approved data handling before sharing.

Table 5. Quality attributes and deployment implications.

SOURCE: MAIN.PY; DNABERT_MODEL.PY; APP.PY; VALIDATION_PIPELINE.PY. NEXT CONTROLS ARE PROPOSED, NOT IMPLEMENTED.

OPERATIONAL PREREQUISITES

The dependency manifest includes NetworkX, NumPy, PyTorch, Transformers, tokenizers, scikit-learn, Flask and Biopython. SMOTE additionally requires imbalanced-learn; XGBoost and LightGBM are optional. No fixed hardware requirement is established in the source, so memory, GPU capacity and run time must be measured rather than invented. Pretrained downloads require connectivity, while browser CDN dependencies can affect offline demonstrations.

REPRESENTATIVE LOCAL ENTRY POINTS

```
python app.py
python gene_classifier.py --model all --kmer 4
python main.py --gfa verified.gfa --labels verified_labels.csv --train --output-dir
trained_model_run1
python main.py --gfa verified.gfa --model trained_model_run1 --kmer 3
```

ENTRY POINTS AND ARGUMENTS IN APP.PY, GENE_CLASSIFIER.PY AND MAIN.PY. VERIFIED INPUT NAMES ARE PLACEHOLDERS, NOT BUNDLED VALIDATED DATASETS.

DESIGN DECISIONS

9. RATIONALE AND EVOLUTION ROADMAP

Decision	Rationale	Trade-off
Keep two execution lanes	Reflects the implemented project and isolates model dependencies.	No unified prediction interface or automatic shared cohort.
Contract SCCs before traversal	Provides a finite DAG for simple path extraction.	Can lose valid sequence order and cycle multiplicity.
Use checkpoint token IDs	Maintains alignment with pretrained embedding rows.	k-mer and checkpoint configurations must match.
Fit classical steps inside CV	Prevents validation records from fitting learned preprocessing.	Reference-based features still assume aligned sequences.
Select before final testing	Reduces test-driven model and threshold selection.	Repeated research decisions can still overfit a reused holdout.

Table 6. Architecture decisions and their explicit costs.

SOURCE: IMPLEMENTATION REVIEW OF GRAPH_PROCESSING.PY, TOKENIZER_MODULE.PY, MAIN.PY AND VALIDATION_PIPELINE.PY.

PRIORITIZED FUTURE ARCHITECTURE

1. Researcher and mentor: establish authoritative labels, coordinate systems, gene verification and source/family identifiers; resolve conflicting annotations before rerunning comparisons.
2. Project developer: preserve orientation and overlap semantics, use biologically valid paths, and introduce variant-centred windows with traceable path-to-label mappings.
3. Project developer: add provenance-aware grouped splits and enforced contamination gates to both workflows; persist classical run metadata and model artifacts.

4. Project developer: only then add a unified experiment interface, background job execution and isolated model sessions; keep endpoint and label semantics explicit.
5. Researcher: evaluate on an independent cohort, add uncertainty intervals and calibration studies, and perform controlled feature/graph ablations before claiming improvement.

> The roadmap is proposed work. No Azure services, microservices, database, graph neural network, automatic ClinVar coordinate mapping or clinical decision support system is claimed as implemented.

ARCHITECTURAL CONTRIBUTION

The defensible project contribution is the implementation of a modular graph-to-language-model preprocessing path alongside a fold-contained classical ML experimentation interface. The architecture supports investigation of alternative genomic representations and makes evaluation boundaries visible. Whether either representation improves biological prediction remains an empirical question, not an architecture conclusion.

TRACEABILITY

10. IMPLEMENTATION MAP AND EVALUATION CHECKLIST

Source module	Architectural responsibility
main.py / utils.py	Graph pipeline orchestration, configuration, split/threshold policy, experiment record.
graph_processing.py	GFA records, directed graph and SCC contraction.
path_extraction.py	Capped DFS enumeration, seeded random walks and path deduplication.
sequence_processing.py / tokenizer_module.py	DNA linearization, checkpoint-compatible k-mers and fixed-length encoding.

Source module	Architectural responsibility
annotation_loader.py / dnabert_model.py	Indexed annotations; BERT loading, fine-tuning, inference and metrics.
app.py / templates/index.html	Web routes, visual dashboard, upload validation and process-local state.
gene_classifier.py	FASTA loading, engineered sequence features, classifier factory and CLI.
validation_pipeline.py	Fold-fitted features, scaling, optional selection/SMOTE, CV selection and holdout reporting.

Table 7. Source-to-architecture traceability.

SOURCE: LOCAL SOURCE MODULES INSPECTED FOR THIS DOCUMENT; NO SOURCE FILES WERE MODIFIED.

MENTOR DEMONSTRATION CHECKLIST

- Explain both input contracts and why the two class vocabularies cannot be silently equated.
- Demonstrate dataset audit before web training, inspect split metadata, then predict a valid full-length FASTA record.
- Show DNABERT graph/path/token stages and a fresh experiment directory with verified matching annotations.
- Show separation of training, threshold/model selection and final testing; explain exact-duplicate versus biological-family leakage.
- Present only reproducible measured results; disclose truncation, graph simplifications, label provenance and clinical limitations.

VERIFICATION BOUNDARY

Existing regression suites are test_dnabert_pipeline.py and test_validation_pipeline.py. They cover evaluation separation, encoded-input grouping, checkpoint safeguards, a tiny local BERT save/reload cycle, fold-contained preprocessing, optional SMOTE boundaries, audits and upload validation. They are software regression checks, not biological benchmarks. This deliverable was checked for document structure, figure inclusion and rendered layout; application tests were not rerun because no application code changed.